

context.md

AI PPT Generator

Project Vision

Build a production-grade AI-powered PowerPoint generation tool that uses the Groq API to generate professional presentations from user-provided topics. The system should create visually appealing PPTX files using predefined design templates while intelligently incorporating user-selected images into appropriate slides.

The application should focus on presentation quality, consistency, maintainability, and extensibility.

Objectives

The system must:

- Generate complete presentations from a topic.
 - Use Groq API for content planning and slide generation.
 - Support multiple professional templates.
 - Allow users to upload and select images.
 - Pass selected image names to Groq during content generation.
 - Let Groq decide where images should appear.
 - Automatically place images on the correct slides.
 - Generate export-ready PPTX files.
 - Maintain strict coding standards and naming conventions.
-

Functional Requirements

Topic Input

Users can provide:

- Topic name
- Detailed prompt
- Number of slides
- Presentation type

Examples:

- Academic Presentation
- Business Presentation
- Project Report
- Seminar Presentation
- Research Presentation
- Product Pitch

Template Selection

The application must provide multiple templates.

Professional

Suitable for business presentations.

Modern

Clean layouts with bold typography.

Academic

Ideal for educational content.

Creative

Visual-heavy design.

Minimal

Simple and elegant design.

Image Management

Upload Images

Users can upload one or more images.

Supported formats:

- PNG
- JPG
- JPEG

- WEBP

Image Selection

Users can choose which uploaded images should be available for the presentation.

Image Metadata

Store:

- Original filename
- Display name
- File path
- Dimensions
- Upload timestamp

Example:

```
{  
  "image_name": "ai_robot.png",  
  "path": "/uploads/ai_robot.png"  
}
```

Groq Integration

Prompt Construction

Every Groq request must include:

User Topic

Example:

```
Artificial Intelligence
```

Slide Count

Example:

```
10
```

Selected Template

Example:

Modern Professional

Selected Images

Example:

robot.png
healthcare_ai.jpg
future_technology.webp

Presentation Rules

The prompt must instruct Groq to:

- Create slide-wise content.
- Decide image placement.
- Reference images only by provided names.
- Maintain presentation flow.
- Keep formatting structured.

Expected Groq Output

Groq should return structured JSON.

Example:

```
{
  "presentation_title": "Artificial Intelligence",
  "slides": [
    {
      "slide_number": 1,
      "slide_type": "title",
      "title": "Artificial Intelligence",
      "content": [],
      "images": []
    },
    {
      "slide_number": 2,
```

```
    "slide_type": "content",
    "title": "Introduction",
    "content": [
      "Definition",
      "Applications"
    ],
    "images": [
      {
        "image_name": "robot.png",
        "position": "right"
      }
    ]
  }
]
```

Image Placement System

Groq determines:

- Which image to use
- On which slide
- Placement location

Supported positions:

- left
- right
- center
- top
- bottom
- background

The PPT generation engine must map these positions to predefined slide coordinates.

Slide Types

Supported slide types:

Title Slide

Contains:

- Main title
- Subtitle

Content Slide

Contains:

- Heading
- Bullet points

Two Column Slide

Contains:

- Text on one side
- Image on the other side

Image Focus Slide

Contains:

- Large image
- Supporting text

Comparison Slide

Contains:

- Left section
- Right section

Timeline Slide

Contains:

- Sequential information

Conclusion Slide

Contains:

- Summary
- Key takeaways

Thank You Slide

Contains:

- Thank you message
-

PPT Generation Rules

Layout Consistency

All slides must:

- Follow selected template
- Use same typography
- Use same color palette
- Use same spacing system

Typography

Heading hierarchy:

- Title
- Section Heading
- Content Text

Font sizes must be centrally managed.

Spacing

Use predefined spacing constants.

Example:

```
TITLE_MARGIN  
CONTENT_MARGIN  
IMAGE_MARGIN
```

No hardcoded values throughout the project.

Project Architecture

```
project_root/
|
├─ app/
|   │
|   ├─ services/
|   │   │
|   │   ├─ groq_service.py
|   │   ├─ ppt_service.py
|   │   └─ image_service.py
|   │
|   ├─ templates/
|   │   │
|   │   ├─ professional.py
|   │   ├─ academic.py
|   │   ├─ modern.py
|   │   └─ creative.py
|   │
|   ├─ models/
|   │   │
|   │   ├─ slide.py
|   │   ├─ image.py
|   │   └─ presentation.py
|   │
|   └─ utils/
|       │
|       ├─ constants.py
|       ├─ validators.py
|       └─ helpers.py
|
└─ main.py

├─ uploads/
├─ generated/
├─ assets/
├─ tests/
└─ requirements.txt
```

Naming Conventions

Files

Use snake_case.

Examples:


```
groq_service.py  
ppt_service.py  
image_service.py
```

Variables

Use snake_case.

Good:

```
user_topic  
selected_images  
slide_data  
template_name
```

Bad:

```
userTopic  
SelectedImages  
slideData
```

Functions

Use verbs.

Good:

```
generate_presentation()  
create_slide()  
insert_image()  
apply_template()  
build_prompt()
```

Classes

Use PascalCase.

Good:

```
GroqService  
PresentationBuilder  
ImageManager  
TemplateManager  
PPTGenerator
```

Constants

Use uppercase.

Good:

```
MAX_SLIDES  
DEFAULT_FONT_SIZE  
IMAGE_PADDING
```

Design Principles

Single Responsibility Principle

Every class should have one responsibility.

Example:

- GroqService → AI communication
 - ImageManager → Image handling
 - PPTGenerator → PPT creation
-

Separation of Concerns

Never mix:

- UI logic
- Groq logic
- PPT generation logic
- Template logic

Reusability

Templates must be reusable.

Image placement logic must be reusable.

Prompt generation must be reusable.

Error Handling

Handle:

- Invalid API key
 - Empty topic
 - Missing images
 - Unsupported image formats
 - Corrupted images
 - PPT export failures
 - Groq response parsing failures
-

Performance Requirements

- Fast slide generation.
 - Efficient image processing.
 - Minimal memory usage.
 - Scalable architecture.
 - Support large presentations.
-

Future Enhancements

- Charts and graphs generation
- Speaker notes generation
- PPT editing mode
- Theme customization
- AI-generated diagrams
- AI-generated images
- Presentation narration
- PDF export
- Collaboration support

Success Criteria

A successful implementation should:

1. Generate high-quality PPTs from any topic.
2. Produce visually consistent slides.
3. Correctly place user-selected images.
4. Use Groq-generated image placement instructions.
5. Follow strict naming conventions.
6. Maintain clean architecture.
7. Be easily extensible for future features.
8. Export professional PPTX presentations ready for use.